Here's a **complete explanation** of all variable modes in Python:

---

## 🐍 4 MODES OF VARIABLES IN PYTHON

---

### 1️⃣ NORMAL VARIABLE — x

python

```
# Simply define without any underscore
# Accessible from ANYWHERE

x = 5
name = "Rahul"
age = 25

class MyClass:
    x = 100        # public variable

    def show(self):
        print(self.x)  # accessible inside class

obj = MyClass()
print(obj.x)        # ✅ accessible outside too
print(MyClass.x)    # ✅ accessible directly
```

**Key Points:**

✅ No restrictions

✅ Accessible everywhere:

→ Inside class

→ Outside class

→ From other modules

→ By child classes

✅ Default mode in Python

✅ No special meaning

**Real Example:**

python

```
class Car:
    brand = "Toyota"    # public
    speed = 120         # public

car = Car()
print(car.brand)   # ✅ Toyota
print(car.speed)   # ✅ 120
```

---

### 2️⃣ PROTECTED VARIABLE — _x

python

```
# Single underscore prefix
# Convention → "Hey! Handle with care"
# NOT strictly enforced by Python

class BankAccount:
    def __init__(self):
        self.name    = "Rahul"   # public
        self._balance = 50000    # protected

    def show_balance(self):
        print(self._balance)     # ✅ accessible inside

class SavingsAccount(BankAccount):  # child class
    def show(self):
        print(self._balance)     # ✅ accessible in child

# Outside class
obj = BankAccount()
```

```
print(obj._balance)
# ✅ Python ALLOWS it but shows warning
# ⚠️ Convention says "Don't access directly"
```

**Key Points:**

⚠️ Single underscore = CONVENTION only

⚠️ Python does NOT enforce protection

✅ Accessible inside class

✅ Accessible in child classes

✅ Technically accessible outside too

BUT considered BAD PRACTICE

📌 It's a GENTLEMAN'S AGREEMENT:

"I'm protected, please don't

touch me directly from outside"

**Real World Use:**

python

```
class Employee:
    def __init__(self):
        self.name     = "Priya"   # public
        self._salary  = 75000     # protected
        self._dept    = "IT"      # protected

    def get_salary(self):
        return self._salary       # access via method

emp = Employee()
print(emp.name)         # ✅ Fine — public
print(emp._salary)      # ⚠️ Works but bad practice
print(emp.get_salary())  # ✅ Correct way to access
```

---

### 3️⃣ PRIVATE VARIABLE — __x

python

```
# Double underscore prefix
# NAME MANGLING happens here
# Python RENAMES it internally

class BankAccount:
    def __init__(self):
        self.name      = "Rahul"  # public
        self._balance  = 50000    # protected
        self.__pin     = 1234     # private ←

    def show_pin(self):
        print(self.__pin)        # ✅ works inside class

obj = BankAccount()
print(obj.name)      # ✅ Rahul
print(obj._balance)  # ✅ 50000 (with warning)
print(obj.__pin)     # ❌ AttributeError!
                     # Cannot access from outside
```

🔥 **NAME MANGLING — The Secret:**

python

```
# Python internally renames __x to:
# _ClassName__x

class MyClass:
    def __init__(self):
        self.__secret = "hidden"

obj = MyClass()
```

```
# Normal access fails
print(obj.__secret)        # ❌ AttributeError

# But name mangling allows this:
print(obj._MyClass__secret)  # ✅ "hidden"
# This is the REAL internal name Python uses
```

**Private in Inheritance:**

python

```
class Parent:
    def __init__(self):
        self.__private = "I am private"

class Child(Parent):
    def show(self):
        print(self.__private)    # ❌ AttributeError!
        # Private NOT accessible in child class

        print(self._Parent__private) # ✅ via mangling

c = Child()
c.show()
```

**Key Points:**

✅ Accessible ONLY inside the class

❌ NOT accessible outside the class

❌ NOT accessible in child classes

🔄 Python applies NAME MANGLING:

  __x → _ClassName__x internally

🎯 Used for truly sensitive data:

  passwords, pins, secret keys

---

### 4️⃣ MAGIC VARIABLE — __x__

python

```
# Double underscore on BOTH sides
# Also called:
# → Dunder (Double UNDERscore) variables
# → Magic variables
# → Special variables

# Python uses these INTERNALLY
# for special behaviors and operations
```

**Most Common Magic Variables:**

python

```
class Student:
    def __init__(self, name, age):  # constructor
        self.name = name
        self.age  = age

    def __str__(self):    # string representation
        return f"Student: {self.name}"

    def __len__(self):    # len() behavior
        return self.age

    def __eq__(self, other):  # == behavior
        return self.name == other.name

s1 = Student("Rahul", 20)
s2 = Student("Priya", 22)

print(str(s1))      # Student: Rahul  (uses __str__)
print(len(s1))      # 20              (uses __len__)
```

```
print(s1 == s2)     # False          (uses __eq__)
```

**All Important Magic Variables:**

python

```
# CLASS RELATED
__name__      # name of class/module
__doc__       # documentation string
__module__    # module where class defined
__dict__      # all attributes as dictionary
__class__     # class of the object

# OBJECT LIFECYCLE
__init__      # constructor → object creation
__del__       # destructor  → object deletion
__new__       # creates new instance

# STRING REPRESENTATION
__str__       # str(obj) → human readable
__repr__      # repr(obj) → developer readable

# MATH OPERATIONS
__add__       # + operator
__sub__       # - operator
__mul__       # * operator
__div__       # / operator
__mod__       # % operator

# COMPARISON
__eq__        # == operator
__ne__        # != operator
__lt__        # <  operator
__gt__        # >  operator
```

```
__le__        # <= operator
__ge__        # >= operator

# CONTAINER BEHAVIOR
__len__       # len() function
__iter__      # iteration behavior
__next__      # next() in loop
__contains__  # 'in' operator

# ATTRIBUTE ACCESS
__getattr__   # getting attribute
__setattr__   # setting attribute
__delattr__   # deleting attribute
```

**Real Magic Variable Examples:**

python

```
# __name__ example
print(__name__)
# → "__main__" if run directly
# → module name if imported

# __doc__ example
class Dog:
    """This is a Dog class"""    # docstring
    pass

print(Dog.__doc__)  # This is a Dog class

# __dict__ example
class Person:
    def __init__(self):
        self.name = "Rahul"
```

```
        self.age  = 25

p = Person()
print(p.__dict__)
# {'name': 'Rahul', 'age': 25}

# __add__ example (operator overloading)
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x,
                     self.y + other.y)

p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2      # calls __add__ automatically
print(p3.x, p3.y) # 4, 6
```

---

### 📊 COMPLETE COMPARISON TABLE

| Mode | Syntax | Access Inside | Access Child | Access Outside | Enforced |
|---|---|---|---|---|---|
| Public | x | ✅ Yes | ✅ Yes | ✅ Yes | — |
| Protected | _x | ✅ Yes | ✅ Yes | ⚠️ Yes* | ❌ No |
| Private | __x | ✅ Yes | ❌ No | ❌ No | ✅ Yes |
| Magic | __x__ | ✅ Yes | ✅ Yes | ✅ Yes | Python only |

*Accessible but bad practice

---

### 🔁 ALL 4 TOGETHER — FULL EXAMPLE

python

```python
class BankAccount:
    bank_name = "SBI"           # 1️⃣ public

    def __init__(self, name, balance, pin):
        self.name     = name     # 1️⃣ public
        self._balance  = balance  # 2️⃣ protected
        self.__pin     = pin      # 3️⃣ private

    def __str__(self):          # 4️⃣ magic
        return f"Account: {self.name}"

    def __eq__(self, other):     # 4️⃣ magic
        return self.__pin == other.__pin

    def get_balance(self):       # method to safely
        return self._balance      # access protected

    def verify_pin(self, pin):    # method to safely
        return self.__pin == pin   # access private


# Creating object
acc = BankAccount("Rahul", 50000, 1234)

# 1️⃣ Public access
print(acc.name)           # ✅ Rahul
print(acc.bank_name)      # ✅ SBI

# 2️⃣ Protected access
```

```
print(acc._balance)        # ⚠️ works but bad practice
print(acc.get_balance())   # ✅ correct way → 50000

# 3️⃣ Private access
print(acc.__pin)           # ❌ AttributeError
print(acc.verify_pin(1234)) # ✅ correct way → True

# 4️⃣ Magic variable
print(str(acc))            # ✅ Account: Rahul
print(acc.__str__())       # ✅ Account: Rahul
```

---

🧠 **EASY WAY TO REMEMBER**

x → 🌍 PUBLIC → Everyone can access

_x → 🔒 PROTECTED → Family only (be careful)

__x → 🔐 PRIVATE → Only me (strictly inside)

__x__ → ✨ MAGIC → Python's special powers

---

💬 **INTERVIEW ANSWER**

Q: What are the different modes of variables in Python?

A: Python has 4 modes:

1. Public (x)
   → No underscore, accessible anywhere

2. Protected (_x)
   → Single underscore, convention-based, should only be used within class and subclasses

3. Private (__x)

→ Double underscore, Python applies name mangling (_ClassName__x), strictly for internal class use

4. Magic (__x__)

→ Double underscore on both sides, reserved by Python for special behaviors like __init__, __str__, __len__ etc.

---

💡 **Key Insight: Python doesn't have TRUE private variables like Java. It uses NAME MANGLING to discourage outside access, but a determined programmer can still access them via _ClassName__variable**